

TP 2 - Monolithe Modulaire

Restructuration et Préparation à la Migration

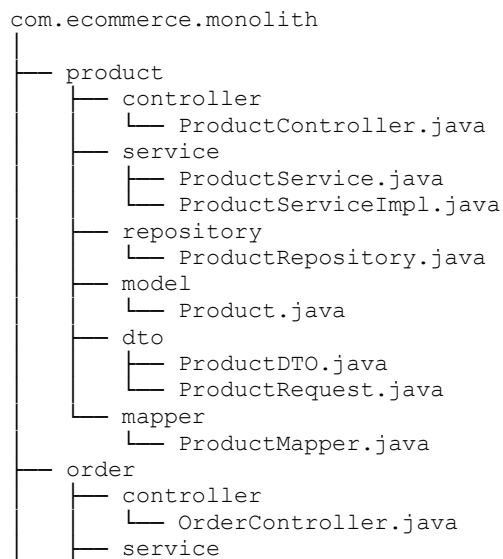
Objectifs

- Restructurer le monolithe en modules métier distincts
- Définir des boundaries claires entre domaines
- Implémenter des interfaces de communication entre modules
- Appliquer les design patterns (DTO, Mapper, Service Layer)
- Préparer le terrain pour une future migration microservices

Outils

- Projet du TP1 comme base
- MapStruct pour le mapping DTO ↔ Entity
- Maven multi-modules (optionnel pour aller plus loin)

Nouvelle Structure



Étapes du TP

Étape 1 : Ajouter MapStruct

Ajoutez la dépendance dans pom.xml depuis le siteweb :

<https://mapstruct.org/documentation/installation/>

Étape 2 : Créer les DTOs

ProductDTO.java

```

@Data
public class ProductDTO {
    private Long id;
    private String name;
    private String description;
    private BigDecimal price;
    private Integer stock;
}

```

CreateProductRequest.java

```

@Data
public class CreateProductRequest {
    @NotBlank(message = "Product name is required")
    private String name;
    private String description;
}

```

```

    @NotNull(message = "Price is required")
    @Positive(message = "Price must be greater than 0")
    private BigDecimal price;
    @PositiveOrZero(message = "Stock cannot be negative")
    private Integer stock = 0;
}

```

Étape 3 : Créer le Mapper

```

@Mapper(componentModel = "spring")
public interface ProductMapper {
    // Convertit une entité Product en DTO
    ProductDTO toDTO(Product product);
    // Convertit une liste d'entités Product en liste de DTO
    List<ProductDTO> toDTOList(List<Product> products);
    // Convertit une requête CreateProductRequest en entité Product
    Product toEntity(CreateProductRequest request);
    // Met à jour une entité Product existante à partir d'une requête
    @Mapping(target = "id", ignore = true)
    void updateEntity(CreateProductRequest request, @MappingTarget Product
product);
}

```

Étape 4 : Refactorer le Service

ProductService.java

```

public interface ProductService {
    // Récupère tous les produits
    List<ProductDTO> getAllProducts();
    // Récupère un produit par son ID
    ProductDTO getProductById(Long id);
    // Crée un nouveau produit
    ProductDTO createProduct(CreateProductRequest request);
    // Met à jour un produit existant
    ProductDTO updateProduct(Long id, CreateProductRequest request);
    // Supprime un produit
    void deleteProduct(Long id);
}

```

ProductServiceImpl.java

```

@Service
@RequiredArgsConstructor
public class ProductServiceImpl implements ProductService {
    private final ProductRepository repository;
    private final ProductMapper mapper;
    @Override
    public List<ProductDTO> getAllProducts() {
        return mapper.toDTOList(repository.findAll());
    }
    @Override
    public ProductDTO getProductById(Long id) {
        Product product = repository.findById(id)
            .orElseThrow(() -> new Exception(
                "Product not found with id: " + id));
        return mapper.toDTO(product);
    }
    @Override
    public ProductDTO createProduct(CreateProductRequest request) {
        Product product = mapper.toEntity(request);
        Product saved = repository.save(product);
        return mapper.toDTO(saved);
    }
    @Override
    public ProductDTO updateProduct(Long id, CreateProductRequest request) {
        Product product = repository.findById(id)
            .orElseThrow(() -> new Exception(

```

```

        "Product not found with id: " + id));

    mapper.updateEntity(request, product);
    Product updated = repository.save(product);
    return mapper.toDTO(updated);
}
@Override
public void deleteProduct(Long id) {
    if (!repository.existsById(id)) {
        throw new Exception(
            "Product not found with id: " + id);
    }
    repository.deleteById(id);
}
}

```

Étape 5 : Adapter le Controller

```

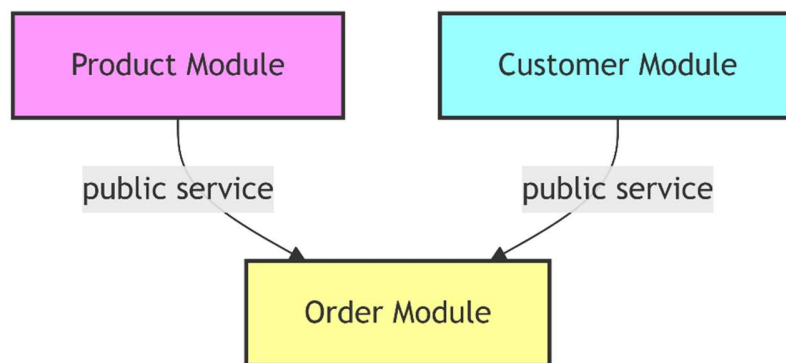
@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {
    private final ProductService productService;
    // Récupérer tous les produits
    @GetMapping
    public ResponseEntity<List<ProductDTO>> getAll() {
        return ResponseEntity.ok(productService.getAllProducts());
    }
    // Récupérer un produit par son ID
    @GetMapping("/{id}")
    public ResponseEntity<ProductDTO> getById(@PathVariable Long id) {
        return ResponseEntity.ok(productService.getProductById(id));
    }
    // Créer un nouveau produit
    @PostMapping
    public ResponseEntity<ProductDTO> create(
        @Valid @RequestBody CreateProductRequest request) {
        ProductDTO created = productService.createProduct(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }
    // Mettre à jour un produit existant
    @PutMapping("/{id}")
    public ResponseEntity<ProductDTO> update(
        @PathVariable Long id,
        @Valid @RequestBody CreateProductRequest request) {
        return ResponseEntity.ok(productService.updateProduct(id, request));
    }
    // Supprimer un produit
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        productService.deleteProduct(id);
        return ResponseEntity.noContent().build();
    }
}

```

Exercice :

Cela un diagramme montrant les dépendances entre modules :

- **Product Module** → Gère uniquement les produits, aucune dépendance externe.
- **Customer Module** → Gère les clients, complètement indépendant.
- **Order Module** → Gère les commandes et dépend des **services publics** de Product et Customer, mais **jamais des repositories directement**.



1. Implémenter les modules Customer et Order avec Entites, Services, Repositories, DTOs et Mapper
2. Ajouter une méthode qui vérifie l'existence du Customer
3. Créer un endpoint pour obtenir l'historique des commandes d'un client